

Predicting Used Car Prices in Belarus

Project report: data cleaning, exploration and decision tree regression

Notebook: final.ipynb | Author: Yash Jain

1. Introduction

Used car prices look random from the outside. Two cars of the same age can be listed at very different prices, and it isn't obvious what people are actually paying for. This project tries to answer a fairly simple question: given the basic details in a listing (year, mileage, engine size, fuel, gearbox, drive type, colour, brand), how well can we predict the asking price? And a follow-up: which of those details matter most?

I used a Kaggle dataset of used car listings from Belarus, did some cleaning and exploration, and trained a decision tree regressor. Everything was done in a single Colab notebook (final.ipynb).

2. Data

The dataset has 56,244 rows and 12 columns. The target is *priceUSD*. The other columns are:

- make and model
- year of production
- condition (with mileage, with damage, for parts)
- mileage in km
- fuel type (petrol, diesel, electric)
- engine volume in cm^3
- colour
- transmission (auto or manual)
- drive unit (front, rear, all-wheel, part-time 4WD)
- segment (body class)

A quick look at the summary statistics showed a few things worth noting. The average price is about \$7,400 but the maximum is \$235,235, so there is a very long tail. The oldest car is from 1910.

Mileage has a maximum of 9,999,999 km, which is obviously a placeholder and not a real odometer reading. Engine volume goes up to 20,000 cm^3 , also suspicious. So the data needed cleaning before modelling.

3. Preprocessing

Dropped columns. I removed *model* because it has far too many distinct values to be useful without heavy encoding, and *segment* because I didn't plan to use it.

Grouping car makes. There are 96 different makes. Encoding them directly would either blow up the number of columns or give the tree a meaningless numeric ordering, so I grouped them into seven

categories by region of origin and market position: Luxury European, Mainstream European, Russian/Eastern European, Asian, American, Specialty and Other. This made the brand analysis much easier to read, but it is also the part of the project I'm least happy with. Some assignments are debatable (Mazda ended up in "Luxury European", which is wrong), and a few makes I didn't list, like Mercedes-Benz, Subaru and Saab, silently fell into "Other". More on that in the limitations section.

Year filter. Most listings are from after 1980, and the handful of older cars behave very differently, so I kept only cars from 1981 onwards.

Missing values. After the year filter there were 47 missing engine volumes and 1,874 missing drive units. Since that is a small share of the data, I dropped those rows rather than trying to fill them in.

Encoding. The six categorical columns (condition, fuel type, transmission, colour, drive unit, brand group) were label-encoded with scikit-learn's LabelEncoder.

Outliers. I computed z-scores across the whole dataframe and kept only rows where every value was within 3 standard deviations. That takes care of the 9,999,999 km mileage and the 20,000 cm³ engines. It also means the most expensive cars in the dataset are removed, because price was included in the z-score calculation. That is a side effect I should have thought about more (see section 6).

4. Exploratory analysis

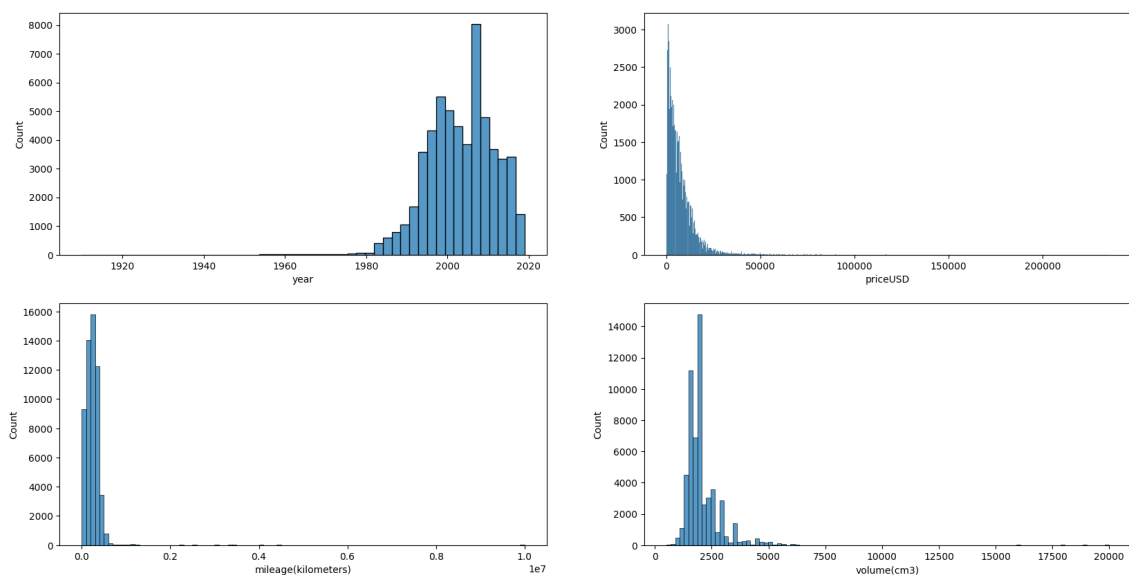


Figure 1. Distributions of year, price, mileage and engine volume before cleaning.

Distributions. Most listings are petrol cars with manual transmission and front-wheel drive, in working condition. Black, silver, blue, white and grey dominate the colour chart. Electric cars are barely present. Most cars were built between the early 1990s and 2019, priced below \$50k, with engines between roughly 1,750 and 2,000 cm³.

Brand. Ranking makes by average price puts Bentley, McLaren, Aston Martin, Tesla and Maserati at the top, which is what you would expect. Some of these makes have only a few listings, so I wouldn't read too much into the exact order.

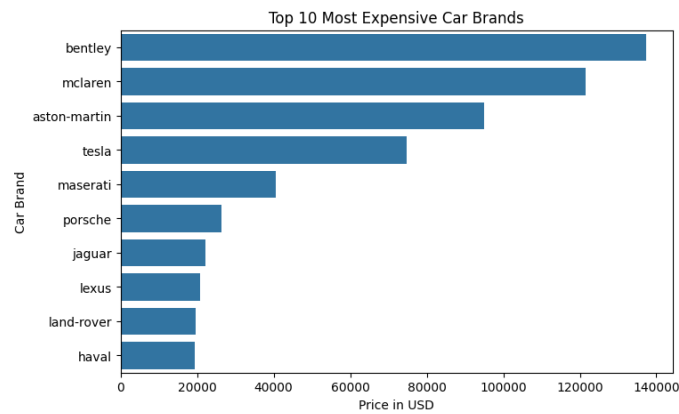


Figure 2. Ten most expensive makes by average price.

Condition. Cars in working condition are the most expensive and their prices rise steadily with year, with a sharp jump for cars from roughly 2015 to 2019. Cars sold for parts stay cheap regardless of year. Damaged cars start out priced close to parts cars for older models but pull away noticeably after about 2000.

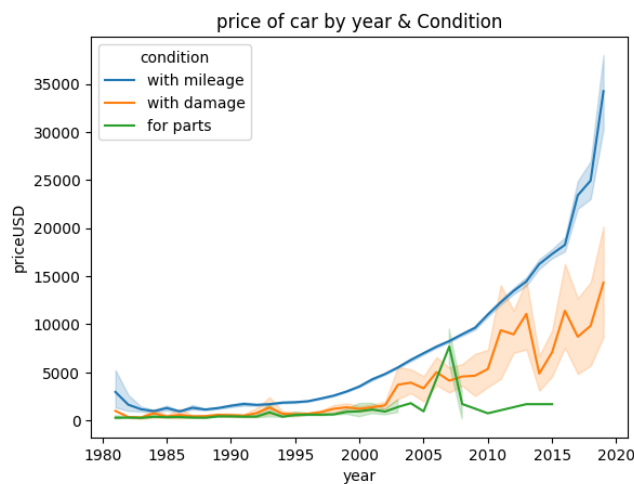


Figure 3. Price by year, split by condition.

Transmission. Automatics are priced above manuals across the whole range, and both climb quickly for cars newer than 2000. **Fuel type.** Petrol and diesel are close until around 2005. After that petrol cars pull ahead, especially after 2015, while diesel prices rise only slightly. Electric cars show up in the data in the mid-1990s and become the most expensive fuel category after 2015.

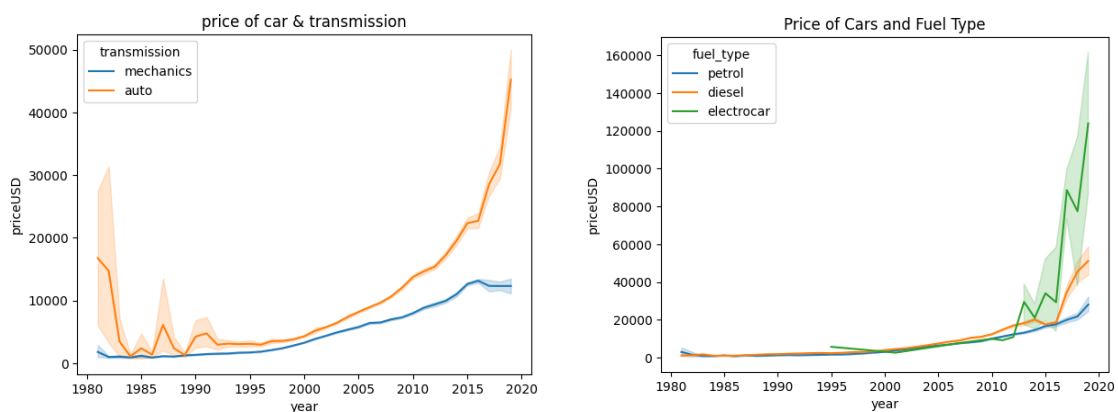


Figure 4. Price by year for transmission type (left) and fuel type (right).

Drive unit. Up to the mid-1990s there is little difference between drive types. After that, front-wheel drive prices rise more slowly than the others, and all-wheel drive becomes the most expensive, followed by part-time 4WD and then rear-wheel drive. **Brand group.** Prices rise sharply after 2005 for most groups. Specialty brands rise the most, followed by Luxury European, American, Asian and Mainstream European. Russian/Eastern European cars are the cheapest and rise the slowest.

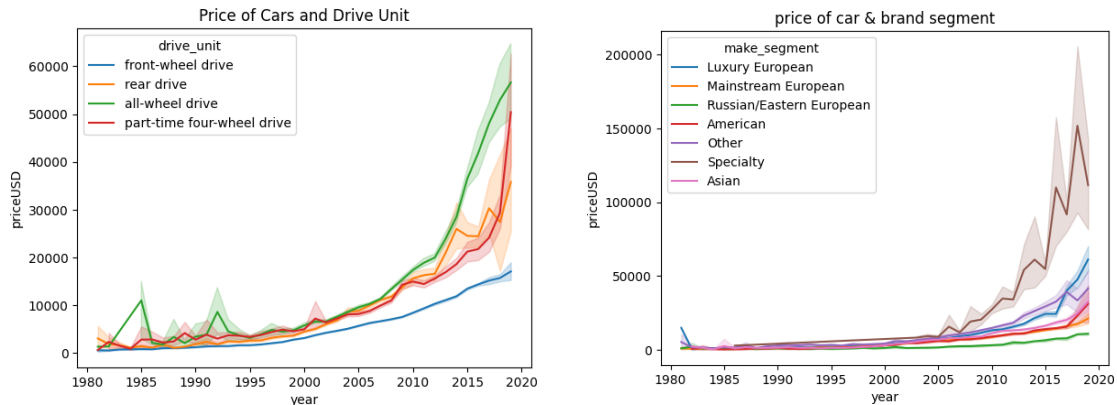


Figure 5. Price by year for drive unit (left) and brand group (right).

Overall the picture is what you would expect: newer, automatic, all-wheel-drive, petrol or electric cars from premium brands cost more.

5. Modelling

I split the cleaned data 80/20 into training and test sets (`random_state=42`) and trained a `DecisionTreeRegressor`. I chose a tree mainly because it is easy to inspect and gives feature importances for free, which matched the second goal of the project.

Hyperparameters were tuned with `GridSearchCV`, 5-fold cross-validation, over `max_depth` (2, 4, 6, 8), `min_samples_split` (2, 4, 6, 8), `max_features` (auto, sqrt, log2) and `random_state` (0, 42). That is 96 combinations, or 480 fits. The best setting came out as `max_depth=8`, `max_features=sqrt`, `min_samples_split=2`. For the final model I used those values plus `min_samples_leaf=4` and `random_state=0`.

Results

Metric	Value
R^2 (train)	0.788
R^2 (test)	0.770
Mean absolute error	\$1,869
Root mean squared error	\$2,712
Mean squared error	7,355,859

The train and test scores are close, so overfitting is not the main problem. With a max depth of 8 the tree is fairly shallow, and I suspect it is leaving some accuracy on the table. An error of about \$1,870 on a car with a median price of roughly \$5,000 is not small in relative terms, especially for cheaper cars.

Feature importance

Feature	Importance
year	0.387
transmission	0.321
mileage	0.168
engine volume	0.053
fuel type	0.042
brand group	0.015
drive unit	0.008
colour	0.006
condition	0.000

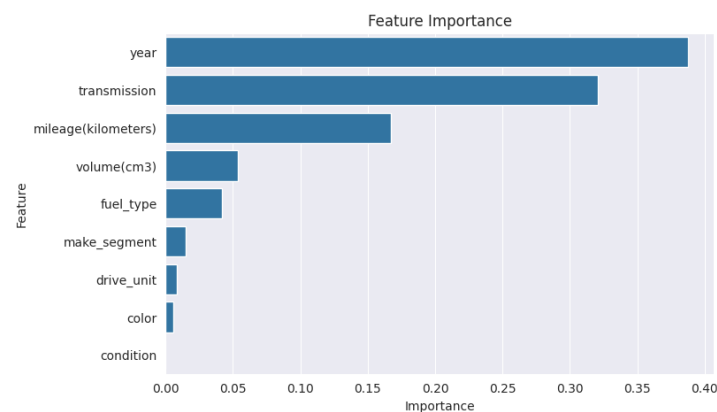


Figure 6. Feature importances from the final tree.

Year is the strongest single predictor, which is not surprising. Transmission coming second was a bit of a surprise to me. My guess is that it is partly acting as a stand-in for other things, since automatics in this market skew towards newer and more expensive cars. Mileage is third. Colour and condition contribute almost nothing.

6. Limitations

I want to be upfront about the weak spots, since a few of them affect how much to trust the numbers.

- **Brand grouping is sloppy.** Mazda, MG, Rover, Peugeot and Chrysler are all in the "Luxury European" list, and Chrysler also appears in the American list (the first match wins, so it never counts as American). Mercedes-Benz, Subaru and Saab were never mentioned and ended up in "Other". Brand group has an importance of only 0.015, so fixing this may not change the score much, but it does undermine the brand-related plots.
- **Hyperparameter tuning was not clean.** The final model uses `min_samples_leaf=4` and `random_state=0`, neither of which came out of the grid search. I hand-picked them after seeing the results. Also, `max_features="auto"` is no longer valid in recent scikit-learn, so 160 of the 480 fits failed and were scored as NaN.

- **Outlier removal also trims the price range.** Applying the z-score filter to every column, including the target, removes the very expensive cars. The model is therefore trained on the "normal" part of the market and probably won't predict supercar prices well.
- **Label encoding on unordered categories.** Colour and brand group are nominal, but label encoding gives them an implied order. Trees cope with this better than linear models, but one-hot encoding would be cleaner.
- **Asking prices, not sale prices.** The data comes from listings, so it tells us what sellers want, not what buyers end up paying.

7. Conclusion

A tuned decision tree can explain about 77% of the variation in used car prices in this dataset using only basic listing details, with a typical error under \$2,000. Year of manufacture and transmission type carry most of the predictive weight, followed by mileage. The exploratory analysis backs this up: prices rise steeply for newer cars, automatics cost more than manuals, petrol and electric cars pull away from diesel after 2015, and all-wheel drive commands a premium.

For a first pass this is decent, but I don't think a single tree is the right final model.

8. Future work

- Fix the brand grouping, or use target encoding on the raw make instead.
- One-hot encode the nominal features.
- Try random forest, gradient boosting or XGBoost on the same split and compare.
- Predict $\log(\text{price})$ to deal with the skewed target, and report relative errors (MAPE) as well.
- Only remove outliers from the input features, and handle price separately.
- Bring back the model column in some compact form, since a Golf and a Passat from the same year are not priced alike.
- Use nested cross-validation so the reported score is not tied to one split.

9. Tools used

Python, pandas, NumPy, Matplotlib, Seaborn, SciPy, scikit-learn, Google Colab.